Folder src

7 printable files

(file list disabled)

src\Addition.java

```java
public class Addition extends Operation {

    /**
     * Addition two numbers
     * @param a is the left value
     * @param b is the right value
     * @param modulo is the matrix modulo
     * @return the result of the addition
     */
    @Override
    public int apply(int a, int b , int modulo) {
        return Math.floorMod(a + b, modulo);
    }

}
```

src\Main.java

```java
public class Main {
    public static void main(String[] args) {
        if(args.length < 5){
            throw new RuntimeException("Not enough arguments");
        }

        int n1 = Integer.parseInt(args[0]),
            m1 = Integer.parseInt(args[1]),
            n2 = Integer.parseInt(args[2]),
            m2 = Integer.parseInt(args[3]),
            modulo = Integer.parseInt(args[4]);

        Matrix matrix1 = new Matrix(n1, m1, modulo),
               matrix2 = new Matrix(n2, m2, modulo);

        System.out.println("Modulo is : " + modulo);

        System.out.println("one ");
        matrix1.printMatrix();

        System.out.println("two ");
        matrix2.printMatrix();


        Matrix add = Matrix.applyOperation(matrix1, matrix2, new Addition()),
               sub = Matrix.applyOperation(matrix1, matrix2, new Subtraction()),
               mul = Matrix.applyOperation(matrix1, matrix2, new Multiplication());

        System.out.println("one + two");
        add.printMatrix();

        System.out.println("one - two");
        sub.printMatrix();

        System.out.println("one * two");
        mul.printMatrix();
```

```
    }
}
```

src\Matrix.java

```java
import java.util.Random; //rand.nextInt

public class Matrix {
    private final int[][] matrix;
    private final int modulo;
    private final int row;
    private final int column;

    /**
     * Matrix Constructor filling the matrix of size row x columns with random values under modulo
     * @param row is the number of rows of the matrix
     * @param column is the number of columns of the matrix
     * @param modulo is the limit that the values of the matrix cannot equal or exceeds
     */
    public Matrix(int row, int column, int modulo) {
        if(row <= 0 || column <= 0 || modulo <= 0){
            throw new RuntimeException("Parameters used to initialize matrix cannot be inferior or equal to zero.\n");
        }
        this.matrix = new int[row][column];
        this.modulo = modulo;
        this.row = row;
        this.column = column;

        for(int i = 0; i < row; i++) {
            for(int j = 0; j < column; j++) {
                // Create an instance of Random class
                Random rand = new Random();
                this.matrix[i][j] = rand.nextInt(modulo);
            }
        }
    }


    /**
     * Matrix Constructor creating a matrix from a modulo and given values
     * @param matrix is the int[][] containing the values
     * @param modulo is the limit that the values of the matrix cannot equal or exceeds
     */
    public Matrix(int[][] matrix, int modulo) {
        if(modulo <= 0){
            throw new RuntimeException("Modulo cannot be inferior or equal to zero.\n");
        }

        if(matrix.length == 0){
            throw new RuntimeException("The matrix cannot be empty.\n");
        }

        this.modulo = modulo;
        this.row = matrix.length;
        this.column = matrix[0].length;
        this.matrix = new int[this.row][this.column];


        for(int i = 0; i < row; i++) {
            if(matrix[i].length == 0){ throw new RuntimeException("The matrix cannot have an empty row.\n"); }
            if(i > 0 && matrix[i].length != matrix[i - 1].length){ throw new RuntimeException("Number of values by rows
must be similar.\n"); }
            this.matrix[i] = new int[column];
            for(int j = 0; j < column; j++) {
               if(matrix[i][j] < 0 || matrix[i][j] >= modulo){
                   throw new RuntimeException("Element (" + i + ", " + j + ") must be in range [0, " + modulo + "[.\n");
```

```java
            }

            this.matrix[i][j] = matrix[i][j];
        }
    }
}


/**
 * Getter to access matrix
 * @return the matrix
 */
public int[][] getMatrix(){
    return matrix;
}


/**
 * Getter to access modulo
 * @return the modulo
 */
public int getModulo(){
    return modulo;
}


/**
 * Print the matrix
 */
public void printMatrix(){
    for(int i = 0; i < row; i++) {
        for(int j = 0; j < column; j++) {
            System.out.print(matrix[i][j] + " ");
        }
        System.out.println();
    }
    System.out.println("\n");
}


/**
 * Resize a matrix before using it on an operation
 * @param A is the matrix to resize
 * @return the new matrix resized
 */
private static Matrix resizeMatrix(Matrix A, int newRow, int newColumn){

    Matrix newA = new Matrix(newRow, newColumn, A.modulo);
    for(int i = 0; i < newRow; i++) {
        for (int j = 0; j < newColumn; j++) {
            if (i < A.row && j < A.column) {
                // Copy existing value
                newA.matrix[i][j] = A.matrix[i][j];
            } else {
                // Filling missing value with zeros
                newA.matrix[i][j] = 0;
            }
        }
    }
    return newA;
}


/**
 * Apply the wanted operations to the matrix
 * @param A is the first matrix
```

```java
     * @param B is the second matrix
     * @return the result of the operation
     */
    public static Matrix applyOperation(Matrix A, Matrix B, Operation operation){

        if(A.modulo != B.modulo){ throw new RuntimeException("Matrix modulo does not match.\n"); }

        //check if the size of the matrix is correct before applying any operation
        int newRow = Math.max(A.row, B.row);
        int newColumn = Math.max(A.column, B.column);

        if (A.row < newRow || A.column < newColumn) {
            A = resizeMatrix(A, newRow, newColumn);
        }
        if (B.row < newRow || B.column < newColumn) {
            B = resizeMatrix(B, newRow, newColumn);
        }

        Matrix result = new Matrix(Math.max(A.row, B.row), Math.max(A.column, B.column), A.modulo);

        for(int i = 0; i < result.row; i++) {
            for(int j = 0; j < result.column; j++) {
                int aValue = (i < A.row && j < A.column) ? A.matrix[i][j] : 0;
                int bValue = (i < B.row && j < B.column) ? B.matrix[i][j] : 0;
                result.matrix[i][j] = operation.apply(aValue, bValue, A.modulo);
            }
        }
        return result;
    }

}
```

src\Multiplication.java

```java
public class Multiplication extends Operation {

    /**
     * Multiply two numbers
     * @param a is the left value
     * @param b is the right value
     * @param modulo is the matrix modulo
     * @return the result of the multiplication
     */
    @Override
    public int apply(int a, int b, int modulo) {
        return Math.floorMod(a * b, modulo);
    }

}
```

src\Operation.java

```java
public abstract class Operation {

    /**
     * Performs the operation, this is an abstract method
     * @param a is the left value
     * @param b is the right value
     * @param modulo is the matrix modulo
     * @return the result of the operation
     */
    public abstract int apply(int a, int b, int modulo);

}
```

src\Subtraction.java

```java
public class Subtraction extends Operation {

    /**
     * Subtraction between two number
     * @param a is the left value
     * @param b is the right value
     * @param modulo is the matrix modulo
     * @return the result of the subtraction
     */
    @Override
    public int apply(int a, int b, int modulo) {
        return Math.floorMod(a - b, modulo);
    }

}
```

src\Test.java

```java
import java.util.Objects;

public class Test {

    /**
     * Function grouping all the function test of the program. Allows to try all the test from one function.
     * @param args entered
     */
    public static void main(String[] args) {
        testInstruction();
        testModuloNotSimilar();

        testConstructorWithNegativeRows();
        testConstructorWithNegativeColumns();
        testConstructorWithNegativeModulo();
        testConstructorWithNegativeGivenValues();
        testConstructorWithEmptyGivenValues();
        testConstructorWithEmptyRowInGivenArray();
        testConstructorWithDifferentRowSizes();
        testConstructorWithValueBiggerThanModulo();
    }

    /**
     * Test given in the instructions
     */
    public static void testInstruction() {
        int[][] one = {{1, 3, 1, 1}, {3, 2, 4, 2}, {1, 0, 1, 0}},
                two = {{1, 4, 2, 3, 2}, {0, 1, 0, 4, 2}, {0, 0, 2, 0, 2}};

        Matrix  m1 = new Matrix(one, 5),
                m2 = new Matrix(two, 5);

        int[][] expectedAdd = {{2, 2, 3, 4, 2}, {3, 3, 4, 1, 2}, {1, 0, 3, 0, 2}},
                expectedSub = {{0, 4, 4, 3, 3}, {3, 1, 4, 3, 3}, {1, 0, 4, 0, 3}},
                expectedMul = {{1, 2, 2, 3, 0}, {0, 2, 0, 3, 0}, {0, 0, 2, 0, 0}};

        System.out.println("The modulus is " + m1.getModulo() + "\none:");
        m1.printMatrix();

        System.out.println("two:");
        m2.printMatrix();

        try{
            Matrix  add = Matrix.applyOperation(m1, m2, new Addition()),
                    sub = Matrix.applyOperation(m1, m2, new Subtraction()),
                    mul = Matrix.applyOperation(m1, m2, new Multiplication());

            System.out.println("one + two:");
```

```java
        add.printMatrix();
        System.out.println("one - two:");
        sub.printMatrix();
        System.out.println("one x two:");
        mul.printMatrix();

        int[][] resultAdd = add.getMatrix(),
                resultSub = sub.getMatrix(),
                resultMul = mul.getMatrix();

        for (int i = 0; i < expectedAdd.length; i++) {
            for (int j = 0; j < expectedAdd[i].length; j++) {
                if(resultAdd[i][j] != expectedAdd[i][j]){
                    throw new RuntimeException("Addition failed");
                }
                if(resultSub[i][j] != expectedSub[i][j]){
                    throw new RuntimeException("Subtraction failed");
                }
                if(resultMul[i][j] != expectedMul[i][j]){
                    throw new RuntimeException("Multiplication failed");
                }
            }
        }
    }catch(RuntimeException e){
        System.err.println("Test, failed. Error: " + e.getMessage());
    }
}

/**
 * Test that the program throw an error due to the matrix modulus not being similar
 */
public static void testModuloNotSimilar() {
    System.out.println("Test, modulo not similar : ");

    Matrix  A = new Matrix(4, 5, 8),
            B = new Matrix(4, 5, 2);

    try{
        Matrix  resultAddition = Matrix.applyOperation(A, B, new Addition()),
                resultSubtraction = Matrix.applyOperation(A, B, new Subtraction()),
                resultMultiplication = Matrix.applyOperation(A, B, new Multiplication());

        // Test failed
        System.out.println("[FAILED], the program should have thrown an error.");
    } catch(RuntimeException e) {
        System.out.println("\t[PASSED], Error: " + e.getMessage());

    }
}

/**
 * Test that the program throws an error when we give the constructor a negative number of rows
 */
public static void testConstructorWithNegativeRows() {
    System.out.println("Test, constructor with a negative number of rows : ");

    try{
        Matrix A = new Matrix(-2, 3, 4);

        // Test failed
        System.out.println("[FAILED], the program should have thrown an error.");
    } catch(RuntimeException e){
        System.out.println("\t[PASSED], Error: " + e.getMessage());
    }
}
```

```java
/**
 * Test that the program throws an error when we give the constructor a negative number of columns
 */
public static void testConstructorWithNegativeColumns() {
    System.out.println("Test, constructor with a negative number of columns: ");

    try{
        Matrix A = new Matrix(2, -3, 4);

        // Test failed
        System.out.println("[FAILED], the program should have thrown an error.");
    } catch(RuntimeException e){
        System.out.println("\t[PASSED], Error: " + e.getMessage());
    }
}


/**
 * Test that the program throws an error when we give the constructor a negative modulo
 */
public static void testConstructorWithNegativeModulo() {
    System.out.println("Test, constructor with modulo negative : ");

    try{
        int[][] values = {{2, 1, 0}, {3, 1, 1}};
        Matrix A = new Matrix(values, -4);

        // Test failed
        System.out.println("[FAILED], the program should have thrown an error.");
    } catch(RuntimeException e){
        System.out.println("\t[PASSED], Error: " + e.getMessage());

    }
}


/**
 * Test that the program throws an error when we give the constructor
 * with a list of value containing a negative value
 */
public static void testConstructorWithNegativeGivenValues() {
    System.out.println("Test, constructor with value given negative : ");

    try{
        int[][] values = {{2, -1, 0}, {-3, 1, 1}};
        Matrix A = new Matrix(values, 4);

        // Test failed
        System.out.println("[FAILED], the program should have thrown an error.");
    } catch(RuntimeException e){
        System.out.println("\t[PASSED], Error: " + e.getMessage());

    }
}


/**
 * Test that the program throws an error when we give the constructor an empty given array of values
 */
public static void testConstructorWithEmptyGivenValues() {
    System.out.println("Test, constructor with empty given array of values : ");

    try{
        int[][] values = {};
        Matrix A = new Matrix(values, 4);
```

```java
        // Test failed
        System.out.println("[FAILED], the program should have thrown an error.");
    } catch(RuntimeException e){
        System.out.println("\t[PASSED], Error: " + e.getMessage());

    }

}


/**
 * Test that the program throws an error when we give the constructor an empty given array of values
 */
public static void testConstructorWithEmptyRowInGivenArray() {
    System.out.println("Test, constructor with an empty row in the given array of values : ");

    try{
        int[][] values = {{3, 2, 1}, {}, {3, 1, 2}};
        Matrix A = new Matrix(values, 4);

        // Test failed
        System.out.println("[FAILED], the program should have thrown an error.");
    } catch(RuntimeException e){
        System.out.println("\t[PASSED], Error: " + e.getMessage());

    }

}


/**
 * Test that the program throws an error when have different sizes of row in the given array of values
 */
public static void testConstructorWithDifferentRowSizes(){
    System.out.println("Test, constructor with different row sizes in the given array of values : ");

    try{
        int[][] values = {{2, 1}, {3, 1, 2}};
        Matrix A = new Matrix(values, 4);

        // Test failed
        System.out.println("[FAILED], the program should have thrown an error.");
    } catch(RuntimeException e){
        System.out.println("\t[PASSED], Error: " + e.getMessage());

    }
}


/**
 * Test that the program throws an error when we give the constructor a value bigger than the modulo
 */
public static void testConstructorWithValueBiggerThanModulo(){
    System.out.println("Test, constructor with given values bigger than modulo : ");

    try{
        int[][] values = {{2, 1, 0}, {3, 1, 6}};
        Matrix A = new Matrix(values, 4);

        // Test failed
        System.out.println("[FAILED], the program should have thrown an error.");
    } catch(RuntimeException e){
        System.out.println("\t[PASSED], Error: " + e.getMessage());

    }
```

```
    }

}
```